SPÉCIFICATION ARCHITECTURALE & FONCTIONNELLE // RÉF. OKOS-VFS-256

LE CUBE 3D DE 256 BITS : MATRICE VFS SPATIALISÉE & NVRAM HOLOGRAPHIQUE

Formalisme mathématique, topologie d'adressage quaternaire et persistance transactionnelle du micro-noyau souverain **Oracle Kernel OS (< 1 Mo)**.

ARCHITECTE : DAVID TREMBLAY-DUVAL DÉPENDANCES : 0 (PYTHON STDLIB) DATE : OCTOBRE 2026

1. Ontologie du Système de Fichiers Virtuel Spatialisé (VFS 3D)

SECTION 01 // TOPOLOGIE

Les systèmes d'exploitation conventionnels reposent sur des graphes acycliques dirigés (arborescences POSIX) qui dissocient la topologie logique des données de leur état d'exécution en mémoire vive. Cette abstraction linéaire engendre une opacité structurelle et une inflation des couches d'indirection. Au sein d'**Oracle Kernel OS**, le stockage et l'état d'exécution convergent dans une structure unifiée : le **Cube de 256 Bits**. Conçu comme une mémoire non volatile holographique (*Holographic NVRAM*), ce système de fichiers virtuel projette l'intégralité de l'état du micro-noyau dans un espace vectoriel euclidien borné, où chaque unité d'information constitue un **voxel adressable** doté de propriétés géométriques, persistantes et exécutables.

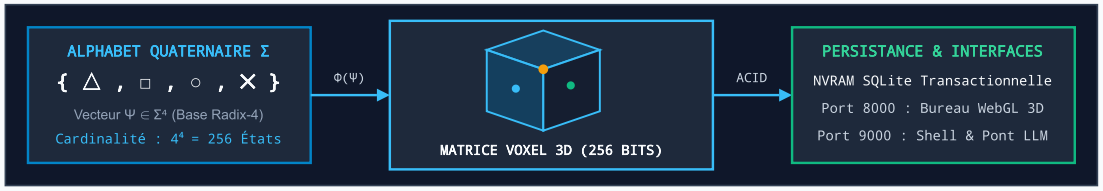


Figure 1.1 — Pipeline d'isomorphisme entre l'espace symbolique quaternaire (Σ^4), le treillis de voxels 3D et le moteur SQLite.

2. Algèbre Quaternaire Symbolique et Bijection d'Adressage

SECTION 02 // FORMALISME

Contrairement à l'algèbre booléenne strictement binaire qui segmente les instructions en séquences linéaires d'octets, le Cube de 256 bits opère sur un alphabet formel quaternaire défini par l'ensemble discret **Σ = { △, □, ○, ✕ }**, associé aux valeurs de base radix-4 `{0, 1, 2, 3}`. Toute adresse ou instruction élémentaire au sein du noyau est gouvernée par un quadruplet symbolique **$\Psi = (s_3, s_2, s_1, s_0) \in \Sigma^4$**. Ce vecteur génère un espace d'états orthogonal de cardinalité exacte **$4^4 = 256$ nœuds**, mis en bijection directe avec les coordonnées spatiales tridimensionnelles **(X, Y, Z)** du treillis de rendu WebGL :
Indice Linéaire : $I(\Psi) = 64 \cdot v(s_3) + 16 \cdot v(s_2) + 4 \cdot v(s_1) + v(s_0) \in [0, 255]$ | **Projection 3D :** $\Phi(I) \rightarrow (X = I \bmod 8,\ Y = \lfloor I / 8 \rfloor \bmod 8,\ Z = \lfloor I / 64 \rfloor)$.

3. Anatomie du Registre de Voxel (256 Bits) & Sémantique SECTION 03 // REGISTRES

Au-delà de la topologie globale à 256 états de l'arbre quaternaire, chaque cellule élémentaire du Cube (voxel) encapsule un **mot d'état atomique de 256 bits (32 octets)** subdivisé en quatre quadrants de 64 bits. Chaque quadrant est régi par l'un des quatre opérateurs symboliques fondamentaux. Cette architecture symétrique permet au micro-noyau de lire, en un seul cycle d'accès mémoire, les coordonnées topologiques du bloc, son pointeur d'inode persistant, son masque d'observabilité temps réel et son vecteur d'exécution dynamique :

Tranche (Bits)	Opérateur Σ	Sous-Système VFS 3D	Fonction Opérationnelle & Couplage Noyau
`Bits [000–063]` (64 bits / 8 B)	`△ (Delta)` Flux & Vecteur	**En-tête Topologique Spatial**	Encode les coordonnées `(X, Y, Z)`, l'indice de couche `I(Ψ)` et les pointeurs de contiguïté vers les voxels adjacents.
`Bits [064–127]` (64 bits / 8 B)	`□ (Carré)` Structure & Socle	**Descripteur d'Inode NVRAM**	Assure le mappage direct vers la page physique de la base SQLite locale, le décalage d'adresse et les drapeaux d'allocation.
`Bits [128–191]` (64 bits / 8 B)	`○ (Cercle)` Observabilité	**Télémétrie & État Thermique**	Stocke l'horodatage de mutation, le niveau d'activité (projeté en chrominance WebGL sur le port `8000`) et le verrou d'accès.
`Bits [192–255]` (64 bits / 8 B)	`✕ (Croix)` Exécution	**Vecteur d'Instruction & Preuve**	Contient l'empreinte d'intégrité du bloc et le pointeur de rappel destiné au démon d'auto-évolution (`SelfEvolutionProtocol`).
Registre Complet (256 bits / 32 B)	`Ψ(△, □, ○, ✕)` Cycle Unifié	**Cellule Holographique Active**	Unité atomique synchronisée entre la mémoire vive Python et le stockage persistant SQLite sans dépendance externe.

4. Moteur de Persistance SQLite & Résilience Hors-Réseau SECTION 04 // PERSISTANCE

Pour éliminer les risques de corruption inhérents aux systèmes de fichiers virtuels purement volatils (RAMDisk), le Cube de 256 bits s'appuie sur une couche de persistance transactionnelle pilotée par le module natif `sqlite3` de la bibliothèque standard. Lors de la compilation du noyau en binaire autonome (`.exe` via PyInstaller), la résolution dynamique des chemins d'exécution ancre la matrice SQLite au niveau du répertoire hôte, garantissant une portabilité intégrale sur support amovible ou terminal isolé.

> **INVARIANT D'ATOMICITÉ ET CONTINUITÉ TACTIQUE (AIR-GAPPED EDGE COMPUTING)**
>
> Toute mutation opérée sur un voxel `Ψ` déclenche une transaction atomique qui met à jour simultanément le tampon mémoire 3D et la table d'état SQLite. En cas de coupure électrique brutale ou d'arrêt matériel sur le terrain, le redémarrage à froid (*Cold Boot*) reconstruit l'intégralité de la géométrie du Cube de 256 bits en quelques millisecondes, sans nécessiter de synchronisation réseau ni d'infrastructure Cloud.

5. Architecture Bi-Portuaire & Observabilité Temps Réel SECTION 05 // INTERFACES

L'un des apports majeurs du Cube de 256 bits réside dans l'abolition de l'effet « boîte noire » des noyaux monolithiques. Le micro-noyau expose l'état de sa matrice tridimensionnelle à travers deux canaux locaux synchrones mais fonctionnellement découplés. Cette séparation permet d'associer une visualisation spatiale immédiate à un contrôle algorithmique de bas niveau :

Bureau Spatial WebGL — PORT 8000

Serveur de rendu asynchrone projetant les 256 voxels sous forme de matrice 3D interactive. Chaque mutation mémoire modifie instantanément la géométrie et la signature lumineuse du voxel ciblé, offrant un diagnostic visuel immédiat de l'entropie du système.

System Shell & Pont LLM — PORT 9000

Canal d'orchestration bas niveau et passerelle d'intelligence locale. Permet d'interroger la matrice par coordonnées quaternaires `(△, □, ○, ×)`, d'inspecter les registres de 256 bits et d'injecter des séquences d'instructions sans latence graphique.

Protocole Auto-Évolutif — DAEMON CORE

Le moteur `SelfEvolutionProtocol` scrute le quadrant d'exécution (×) des voxels actifs pour compiler dynamiquement des modules en mémoire, adapter ses routines internes et faire évoluer l'OS sans rechargement lourd.

Souveraineté & Auditabilité — < 1 MO // 0 DEP

L'absence absolue de bibliothèques tierces élimine les vulnérabilités de chaîne d'approvisionnement logicielle (*Supply-Chain Attacks*). Le code source demeure intégralement auditable par des experts de défense.

6. Cycle d'Exécution Symbolique & Auto-Évolution SECTION 06 // ÉVOLUTION

Lorsque le démon d'introspection `SelfEvolutionProtocol` est sollicité via le port `9000` ou par un événement interne du noyau, il parcourt les vecteurs d'état du Cube selon une séquence déterministe en quatre phases : **(1) Acquisition du flux** sur le segment `△`, **(2) Ancrage structurel** et verrouillage SQLite sur le segment `□`, **(3) Validation d'intégrité et d'observabilité** sur le segment `○`, puis **(4) Évaluation et instanciation dynamique** sur le segment `×`. Ce bouclage cybernétique transforme le système de fichiers en une matrice cognitive autonome, capable d'absorber de nouvelles logiques métier tout en maintenant une empreinte mémoire inférieure à 1 Mo.

PROPRIÉTÉ INTELLECTUELLE & LICENCE D'EXPLOITATION EXCLUSIVE
Copyright © 2026 **David Tremblay-Duval**. Tous droits réservés (*All Rights Reserved*). L'architecture d'**Oracle Kernel OS**, le formalisme du Cube VFS 3D de 256 bits, la logique quaternaire associée et le code source afférent constituent la propriété intellectuelle exclusive de l'auteur. Toute reproduction, modification, intégration ou exploitation commerciale/institutionnelle sans autorisation écrite préalable est strictement interdite. Document certifié pour dépôt GitHub et soumission aux guichets d'innovation de défense (IDEeS / AID).